

Transition: DS + Algorithms

Big-O
(really this time!)



First: Worksheet

How did induction go?

Today: Big-O

The why:

Given all the low level things we can't control, can be hard to perfectly count "# of operations".

Big-O ignores constants, to focus on "long term" behavior

Being twice as fast is still good!

But, if $10 n \log n$ versus $10^6 n^2$:

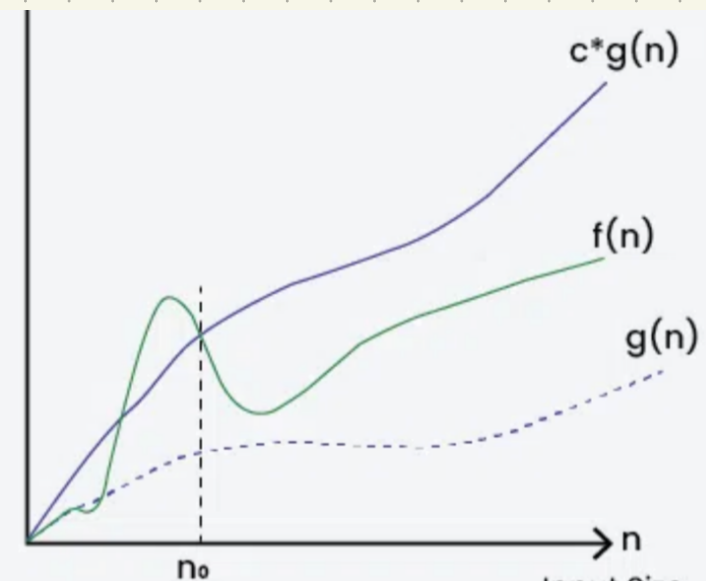
Big-O

Let f and g be functions from

$\mathbb{Z}^+$ to $\mathbb{R}^+$
why?

We say $f(n) = O(g(n))$ if $\exists$
constants c and n_0 s.t. $\forall n > n_0$,
 $f(n) \leq c \cdot g(n)$

Note: $O(g(n))$ is
a set.



Example

$$f(x) = x^2 + 2x + 1 \quad \text{is} \quad O(x^2)$$

proof: Find c and n_0 , then show inequality holds.

General strategy

- Start with n_0 : pick it so can estimate size of $f(n)$ for $n > n_0$.
 - ↳ often based on coefficients

- Then look for a constant c so inequality holds.

Example: Show $f(x) = 5x^2 + 25x + 3$ is $O(x^3)$.

We'll write $f(x) = O(g(x))$.

NOT AN EQUALITY!

• $x^2 + 2x + 1$ is $O(x^2)$

• $x^2 + 2x + 1$ is $O(x^3)$

Really, means $f(x) \in \{ \text{functions that are } O(g(x)) \}$

Polynomials are usually nice $\rightarrow$ but
Can do many functions!

Ex: $\sin x$ is $O(1)$.

Ex: n is $O(n \log_2 n)$.

Ex Show that n^2 is not $O(n)$

proof: harder! need to show

$$\neg (\exists c \exists n_0 \text{ s.t. } \forall n > n_0, n^2 \leq c \cdot n)$$



$\forall c$ and $\forall n_0$, $\exists$ some $n > n_0$ with

So:

That limit definition

I did $f(n) = O(g(n))$ if $\exists n_0 \ \& \ c$ s.t.

$$\forall n > n_0, \quad f(n) \leq c \cdot g(n)$$

Take that inequality:

$$\text{So } f(n) = O(g(n)) \Leftrightarrow$$

Big Ω

Let f and g be functions from $\mathbb{Z}^+ \rightarrow \mathbb{R}^{\geq}$

We say f is $\Omega(g(n))$ if

$\exists$ positive constants c and n_0
such that $\forall n > n_0$,

$$f(n) \geq c \cdot g(n)$$

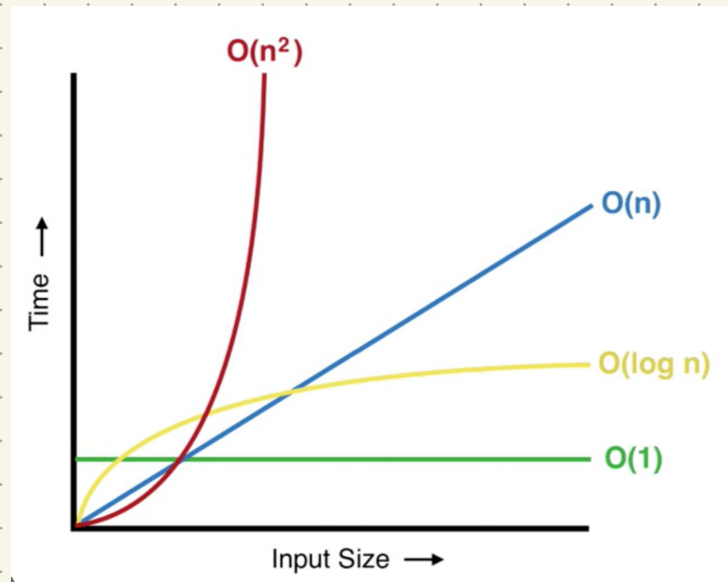
" f is big-omega of g "

Example: $5 \cdot n$ is $\Omega(n)$:

Big- Θ

If f is $O(g)$ and $\Omega(g)$ then f is $\Theta(g)$.

This gives a way to nicely "bucket" function size:



Theorems

Let $p(n)$ be a degree k polynomial, so

$$p(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$$

for constants a_i and $a_k \neq 0$.

Then $p(n) = \Theta(n^k)$.

Proof:

Big- Θ :

Big-O:

Using this theorem;

Give big- Θ estimates for:

- $f(x) = \frac{1}{8} x^5 + x^3 + 2$

- $f(x) = 20000 x^2 - 100000 x$

- $f(x) = \frac{2x^3}{3000} - x + 2$

Other tricks:

Sometimes $f(n) = \sum_{i=1}^n \log_2 n$

How to get Θ -bound?

Another: $\log_2 n$ versus $\log_{10} n$.

What is a log?

Identities:

Another:

$$\sum_{i=1}^n \log_2 i =$$

Next time:

Analyzing algorithms with $\log$ - Θ .